MoA Prediction — Multi-Model Ensemble

Kaggle competition: Mechanisms of Action (MoA) Prediction (https://www.kaggle.com/competitions/lish-moa)

Multi-label classification of 206 biological mechanisms of action from gene-expression and cell-viability profiles.

Workflow

This notebook is organised into the following numbered steps. Section headers and in-code comments are tagged with the matching step number (`# Step N` or `# Step N.k`), so any block of code can be traced back to its place in the pipeline.

1. **Setup** — imports and global configuration
2. **Data loading and cleaning**
3. **Feature engineering** — row statistics, categorical encoding
4. **Model architectures** — TabularResNet, FT-Transformer
5. **Evaluation metric** — mean multi-label binary log loss
6. **Loss variants** — focal loss and label smoothing
7. **Per-fold training routine** — single model, single fold, single seed, with an inner validation split used only for model selection
8. **Per-fold preprocessing pipelines** — RankGauss(+PCA) and RankGauss-only variants
9. **Cross-validation — neural network ensemble** (ResNet, FT-Transformer), multi-seed bagging, 30 jobs scheduled on a pool of parallel workers across GPUs
10. **Third model — CatBoost** on raw (non-PCA) features
11. **Ensemble blending** — nested-CV weight search over the three base models
12. **Experiment logging**
13. **Submission building and sanity checks**
14. **Main pipeline** — runs steps 2–13 end to end and writes `submission.csv`
    - 14.1 load and clean data
    - 14.2 feature engineering
    - 14.3 neural-network cross-validation
    - 14.4 CatBoost cross-validation
    - 14.5 ensemble blending
    - 14.6 experiment logging
    - 14.7 test-set preprocessing and inference
    - 14.8 submission and sanity checks

Known limitations

- The competition data has no drug identifier, so the cross-validation cannot be grouped by compound. The same compound may appear in the training and validation split of a fold; scores can therefore be optimistic relative to the leaderboard.
- Early stopping, learning-rate scheduling and best-epoch selection of the neural networks use an **inner** validation split carved out of each training fold. The outer fold that produces the out-of-fold (OOF) predictions is never used for model selection. CatBoost uses a fixed number of iterations.
- The FT-Transformer is a simplified variant inspired by Gorishniy et al. (2021): post-LayerNorm encoder, GELU feed-forward, no ReGLU.

Note: the outputs in this notebook come from a single clean run (*Restart & Run All*) on Kaggle with 2x NVIDIA T4 GPUs. To reproduce it, see `software/README.md` .

1. Setup: imports and configuration

Installation of the `iterative-stratification` library with `pip`, as it is not included in the standard `scikit-learn` distribution. It provides multi-label stratified cross-validation, which keeps the proportion of positives of all 206 targets balanced across folds.

In [1]:

```
# Step 1.1 - install the multi-label stratification library (quiet mode)
!pip install -q iterative-stratification
```

Import all necessary libraries (PyTorch, Pandas, Scikit-Learn, Joblib) and set the global experiment constants.

In [2]:

```
# Step 1.2 - imports
import os
import glob
import json
import time
import math
import random
import warnings

# Set before importing PyTorch to reduce GPU memory fragmentation
# when multiple parallel processes allocate tensors.
os.environ.setdefault("PYTORCH_CUDA_ALLOC_CONF", "expandable_segments:True")

import numpy as np
import pandas as pd
import joblib
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from sklearn.decomposition import PCA
from sklearn.preprocessing import QuantileTransformer
from sklearn.feature_selection import VarianceThreshold
from sklearn.exceptions import DataConversionWarning
from iterstrat.ml_stratifiers import (MultilabelStratifiedKFold,
                                      MultilabelStratifiedShuffleSplit)
from scipy.optimize import minimize
from joblib import Parallel, delayed
from joblib.externals.loky import get_reusable_executor

# Only a known, harmless warning is silenced; other warnings stay visible.
warnings.filterwarnings("ignore", category=DataConversionWarning)

# Step 1.3 - global configuration
DATA_DIR = "/kaggle/input/competitions/lish-moa"
WORK_DIR = "/kaggle/working"
N_FOLDS = 5
RANDOM_STATE = 42
N_QUANTILES = 100
PCA_G_COMPONENTS = 50       # for 'g-...' components
PCA_C_COMPONENTS = 15       # for 'c-...' components
BATCH_SIZE = 256
EPOCHS = 35
```

```
EPOCHS = 35

LR = 1e-3
WEIGHT_DECAY = 1e-5
CLIP_EPS = 1e-15
CTL_LABEL = "ctl_vehicle"

os.makedirs(WORK_DIR, exist_ok=True)

N_GPUS = torch.cuda.device_count()
DEVICE = torch.device("cuda" if N_GPUS > 0 else "cpu")
USE_AMP = N_GPUS > 0                    # mixed precision (fp16) only if a GPU is available

SEEDS = [42, 202, 2024]                 # multi-seed bagging
EARLY_STOPPING_PATIENCE = 8             # epochs without improvement before stopping
INNER_VAL_FRACTION = 0.1                # share of each training fold held out for model selection
VARIANCE_THRESHOLD = 0.0                # 0.0 = removes only constant columns
USE_FOCAL_LOSS = False                  # True to use focal loss instead of BCE (training only)
FOCAL_ALPHA = 0.25
FOCAL_GAMMA = 2.0
LABEL_SMOOTHING = 0.001                 # 0.0 to disable
LOG_DIR = f"{WORK_DIR}/experiment_logs"

# CatBoost configs
N_ESTIMATORS_CATBOOST = 250
CATBOOST_LEARNING_RATE = 0.08
CATBOOST_DEPTH = 5

# Parallel jobs for the neural networks: 2 per GPU (overlaps compute and
# launch overhead), so both T4s stay busy for the whole CV instead of
# running fold-by-fold, seed-by-seed, model-by-model in strict sequence.
# resnet and ft_transformer both draw from this one pool (2 models x 3
# seeds x 5 folds = 30 jobs, executed by N_JOBS_NN workers at a time).
#
# CatBoost has its own separate, sequential per-fold CV, on features without PCA.
NN_JOBS_PER_GPU = 2
N_JOBS_NN = max(1, N_GPUS * NN_JOBS_PER_GPU)

# The FT-Transformer's batch size is much smaller than the ResNet one
# to avoid Out Of Memory errors: self-attention has quadratic computational
# cost O(L^2) in the number of features (~890 tokens per row).
BATCH_SIZE_BY_MODEL = {"resnet": BATCH_SIZE, "ft_transformer": 64}
INFERENCE_BATCH_SIZE_BY_MODEL = {"resnet": 1024, "ft_transformer": 128}

print(f"GPUs detected: {N_GPUS} | AMP: {USE_AMP} | parallel NN jobs: {N_JOBS_NN}")

torch.backends.cudnn.benchmark = True
```

```
GPUs detected: 2 | AMP: True | parallel NN jobs: 4
```

2. Data loading and cleaning

Definition of the `load_raw_data` and `clean_and_align` functions: import the CSV files, remove null values and duplicates, and align features and targets on `sig_id`.

In [3]:

```
# Step 2.1 - load the four competition CSV files
def load_raw_data(data_dir):
    features = pd.read_csv(f"{data_dir}/train_features.csv")
    targets = pd.read_csv(f"{data_dir}/train_targets_scored.csv")
    test = pd.read_csv(f"{data_dir}/test_features.csv")
    sample_sub = pd.read_csv(f"{data_dir}/sample_submission.csv")
    return features, targets, test, sample_sub


# Step 2.2 - clean and align features/targets
def clean_and_align(features, targets):
    """Drops nulls/duplicates and keeps only the sig_id values present in BOTH
    tables, so a row dropped from one table is always dropped from the other."""
    features = features.dropna(axis=0).drop_duplicates()
    targets = targets.dropna(axis=0).drop_duplicates()

    common_ids = set(features["sig_id"]) & set(targets["sig_id"])
    features = features[features["sig_id"].isin(common_ids)]
    targets = targets[targets["sig_id"].isin(common_ids)]

    features = features.sort_values("sig_id").reset_index(drop=True)
    targets = targets.sort_values("sig_id").reset_index(drop=True)

    # Strict check on perfect alignment between the independent variables and targets
    assert features.shape[0] == targets.shape[0], "Row count mismatch between features and targets"
    assert (features["sig_id"].values == targets["sig_id"].values).all(), "Mismatched sig_id values"
    return features, targets
```

3. Feature engineering

Computation of aggregate statistics for each row and encoding of categorical variables.

In [4]:

```
# Step 3.1 - row-wise summary statistics
def add_row_statistics(df, g_cols, c_cols):
    """
    Sum, mean, standard deviation, kurtosis and skewness are extracted separately
    for the g- and c- feature groups to provide the model with summary information on
    general activation.
    """
    df = df.copy()
    df["g_sum"] = df[g_cols].sum(axis=1)
    df["g_mean"] = df[g_cols].mean(axis=1)
    df["g_std"] = df[g_cols].std(axis=1)
    df["g_kurtosis"] = df[g_cols].kurtosis(axis=1)
    df["g_skew"] = df[g_cols].skew(axis=1)

    df["c_sum"] = df[c_cols].sum(axis=1)
    df["c_mean"] = df[c_cols].mean(axis=1)
    df["c_std"] = df[c_cols].std(axis=1)
    df["c_kurtosis"] = df[c_cols].kurtosis(axis=1)
    df["c_skew"] = df[c_cols].skew(axis=1)
    return df


# Step 3.2 - one-hot encoding of cp_time / cp_dose
def encode_categoricals(df, dummy_cols, reference_columns=None):
    """One-hot encodes cp_time/cp_dose, aligned to the training columns."""
    df = pd.get_dummies(df, columns=dummy_cols, dtype=np.float64)
    if reference_columns is not None:
        df = df.reindex(columns=reference_columns, fill_value=0)
    return df
```

4. Model architectures

Definition of 2 neural networks in PyTorch:

- TabularResNet
- FTTransformer

In [5]:

```
# Step 4.2 - Tabular ResNet
class TabularResNet(nn.Module):
    name = "resnet"

    def __init__(self, input_dim, output_dim=206, hidden_dim=512, dropout_rate=0.2):
        super().__init__()
        self.in_proj = nn.Sequential(
            nn.BatchNorm1d(input_dim),
            nn.Dropout(dropout_rate),
            nn.Linear(input_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
            nn.SiLU(),
        )
```

```
        self.block1 = self._make_block(hidden_dim, dropout_rate)
        self.block2 = self._make_block(hidden_dim, dropout_rate)
        self.out_proj = nn.Sequential(
            nn.SiLU(), nn.Dropout(dropout_rate), nn.Linear(hidden_dim, output_dim)
        )

    @staticmethod
    def _make_block(hidden_dim, dropout_rate):
        return nn.Sequential(
            nn.Dropout(dropout_rate),
            nn.Linear(hidden_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
            nn.SiLU(),
            nn.Dropout(dropout_rate),
            nn.Linear(hidden_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
        )

    def forward(self, x):
        x = self.in_proj(x)
        x = F.silu(x + self.block1(x))
        x = F.silu(x + self.block2(x))
        return self.out_proj(x)


# Step 4.3 - FT-Transformer: feature tokenizer
class FeatureTokenizer(nn.Module):
    """
    Projects each scalar numeric feature into a token vector of dimension
    d_token via an independent linear transform.
    """

    def __init__(self, num_features, d_token):
        super().__init__()
        self.weight = nn.Parameter(torch.empty(num_features, d_token))
        self.bias = nn.Parameter(torch.empty(num_features, d_token))
        nn.init.kaiming_uniform_(self.weight, a=math.sqrt(5))
        nn.init.zeros_(self.bias)

    def forward(self, x):
        # x shape: (Batch, Features)
        # broadcast: (Batch, Features, 1) * (Features, d_token) -> (Batch, Features, d_token)
        return x.unsqueeze(-1) * self.weight + self.bias


# Step 4.4 - FT-Transformer: encoder and head
class FTTransformer(nn.Module):
    """
    Simplified FT-Transformer variant (inspired by Gorishniy et al., 2021).
    Turns the tabular input into a sequence of tokens (one per feature),
    prepends a global [CLS] token, and applies multi-head self-attention.
    Diversity relative to the ResNet comes from the self-attention across features.
    """
```

```
    name = "ft_transformer"

    def __init__(self, input_dim, output_dim=206, d_token=64, n_blocks=2,
                 attention_heads=4, dropout=0.2):
        super().__init__()
        self.tokenizer = FeatureTokenizer(input_dim, d_token)

        # Global [CLS] token
        self.cls_token = nn.Parameter(torch.zeros(1, 1, d_token))
        nn.init.normal_(self.cls_token, std=0.02)

        # Transformer blocks (GELU is natively supported and is the standard choice)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_token,
            nhead=attention_heads,
            dim_feedforward=d_token * 4,
            dropout=dropout,
            activation="gelu",
            batch_first=True,
        )
        # enable_nested_tensor=False: no padding mask is used, and this avoids
        # a PyTorch UserWarning at construction time.
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=n_blocks,
                                                 enable_nested_tensor=False)

        self.layer_norm = nn.LayerNorm(d_token)
        self.head = nn.Sequential(
            nn.Dropout(dropout),
            nn.Linear(d_token, output_dim)
        )

    def forward(self, x):
        # Tokenization: (B, input_dim) -> (B, input_dim, d_token)
        tokens = self.tokenizer(x)

        # Prepend [CLS] token: (B, 1 + input_dim, d_token)
        cls_tokens = self.cls_token.expand(x.size(0), -1, -1)
        x = torch.cat((cls_tokens, tokens), dim=1)

        # Transform via self-attention
        x = self.transformer(x)

        # Extract the output at the [CLS] token position (index 0)
        cls_out = self.layer_norm(x[:, 0, :])

        # Multi-label classification head
        return self.head(cls_out)
```

5. Evaluation metric

In [6]:

```
# Step 5.1 - competition metric
def compute_moa_log_loss(y_true, y_pred, eps=CLIP_EPS):
    """Mean binary log loss across all samples and all 206 targets, clipped
    to avoid log(0)."""
    # float64 is required: in float32, 1 - 1e-15 rounds to exactly 1.0, the clip
    # becomes ineffective and log(0) produces -inf / NaN.
    y_true = np.asarray(y_true, dtype=np.float64)
    y_pred = np.clip(np.asarray(y_pred, dtype=np.float64), eps, 1 - eps)
    loss_matrix = -(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))
    return np.mean(loss_matrix)
```

6. Loss variants: focal loss and label smoothing

Definition of `FocalLossWithLogits` and the `smooth_labels` function. Both affect the **training** loss only: model selection always uses the plain, non-smoothed log loss.

In [7]:

```
# Step 6.1 - focal loss
class FocalLossWithLogits(nn.Module):
    """Multi-label focal loss operating on raw logits (no sigmoid applied
    beforehand). Down-weights "easy" (already well-classified) samples,
    useful when positives are extremely rare, as in the MoA dataset."""

    def __init__(self, alpha=FOCAL_ALPHA, gamma=FOCAL_GAMMA):
        super().__init__()
        self.alpha = alpha
        self.gamma = gamma

    def forward(self, logits, targets):
        bce = F.binary_cross_entropy_with_logits(logits, targets, reduction="none")
        probs = torch.sigmoid(logits)
        pt = probs * targets + (1 - probs) * (1 - targets)
        focal_weight = (1 - pt) ** self.gamma
        alpha_weight = self.alpha * targets + (1 - self.alpha) * (1 - targets)
        return (alpha_weight * focal_weight * bce).mean()


# Step 6.2 - label smoothing
def smooth_labels(y, smoothing):
    """y*(1-eps) + 0.5*eps: prevents the model from being overconfident
    (exact 0/1 targets), useful with strongly imbalanced classes."""
    if smoothing <= 0.0:
        return y
    return y * (1 - smoothing) + 0.5 * smoothing
```

7. Per-fold training routine

Training of one model on a specific fold and seed using:

- **Inner validation split:** a stratified share (`INNER_VAL_FRACTION`) of the training fold is held out for early stopping, learning-rate scheduling and best-epoch selection. The outer validation fold is used **once**, after training, to produce the OOF predictions.

- **Honest validation:** the selection metric is always the plain, non-smoothed log loss, regardless of the training loss (BCE or focal) and of label smoothing.
- **AMP (Mixed Precision):** uses `torch.amp.GradScaler` to speed up the training on GPUs while maintaining stability.
- **Reproducibility:** seeds are set *before* the model is built, so initial weights depend on the seed.
- **In-RAM Checkpoint Saving:** the best model's `state_dict` is cloned to system RAM (CPU) and saved to disk only at the end of training, avoiding slow I/O writes at every improving epoch.

In [8]:

```
# Step 7.1 - global seeding helper
def set_global_seed(seed):
    """Seeds Python, NumPy and PyTorch (CPU and all GPUs)."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)


# Step 7.2 - batched probability prediction
def predict_probabilities(model, X, device, batch_size, use_amp=False):
    """Returns sigmoid probabilities for X, computed in mini-batches."""
    model.eval()
    loader = DataLoader(TensorDataset(torch.tensor(X, dtype=torch.float32)),
                        batch_size=batch_size, shuffle=False)
    prob_chunks = []
    with torch.no_grad():
        for (x_batch,) in loader:
            x_batch = x_batch.to(device, non_blocking=True)
            with torch.autocast(device_type=device.type, dtype=torch.float16, enabled=use_amp):
                logits = model(x_batch)
            prob_chunks.append(torch.sigmoid(logits.float()).cpu().numpy())
    # float64: keeps the later clipping to [1e-15, 1 - 1e-15] effective
    return np.vstack(prob_chunks).astype(np.float64)


# Step 7.3 - train one model on one fold / seed
def train_fold(fold, model, X_train, y_train, X_val, y_val,
               epochs=EPOCHS, batch_size=BATCH_SIZE, lr=LR,
               weight_decay=WEIGHT_DECAY, device=DEVICE, work_dir=WORK_DIR,
               seed=None, checkpoint_suffix="",
               early_stopping_patience=EARLY_STOPPING_PATIENCE,
               use_focal_loss=USE_FOCAL_LOSS, label_smoothing=LABEL_SMOOTHING,
               inner_val_fraction=INNER_VAL_FRACTION,
               use_amp=USE_AMP, verbose=False):
    """Trains one model on one fold/seed with AMP, early stopping, optional
    focal loss and label smoothing.

    Model selection (early stopping, LR scheduler, best epoch) uses an inner
    split of the training fold. The outer validation fold (X_val, y_val) is
    only used after training, to score the selected checkpoint. Returns the
    outer-validation probabilities and the outer-validation log loss.

    `verbose=False` keeps this function silent so that running it 30 times in
    parallel does not flood the notebook output; the calling code prints a
    single aggregated summary per model instead (see Step 9)."""
```

```
seed_value = RANDOM_STATE if seed is None else seed
set_global_seed(seed_value)

device = torch.device(device)
use_amp = use_amp and device.type == "cuda"
model = model.to(device)

# Step 7.3.1 - inner stratified split (model selection only)
inner_splitter = MultilabelStratifiedShuffleSplit(
    n_splits=1, test_size=inner_val_fraction, random_state=seed_value)
fit_idx, inner_idx = next(inner_splitter.split(X_train, y_train))
X_fit, y_fit = X_train[fit_idx], y_train[fit_idx]
X_inner, y_inner = X_train[inner_idx], y_train[inner_idx]

# Step 7.3.2 - data loader (drop_last avoids a size-1 batch breaking BatchNorm)
pin_memory = device.type == "cuda"
shuffle_generator = torch.Generator().manual_seed(seed_value)
train_loader = DataLoader(
    TensorDataset(torch.tensor(X_fit, dtype=torch.float32),
                  torch.tensor(y_fit, dtype=torch.float32)),
    batch_size=batch_size, shuffle=True, drop_last=True,
    pin_memory=pin_memory, generator=shuffle_generator,
)

# Step 7.3.3 - loss, optimiser, scheduler, AMP scaler
criterion = FocalLossWithLogits() if use_focal_loss else nn.BCEWithLogitsLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="min", factor=0.1, patience=3)
try:
    scaler = torch.amp.GradScaler(device="cuda" if use_amp else "cpu", enabled=use_amp)
except TypeError:
    scaler = torch.cuda.amp.GradScaler(enabled=use_amp)

best_inner_loss = float("inf")
best_state_dict = None
epochs_without_improvement = 0
checkpoint_path = f"{work_dir}/best_{model.name}_fold_{fold}{checkpoint_suffix}.pth"

# Step 7.3.4 - epoch loop
for epoch in range(epochs):
    model.train()
    for x_batch, y_batch in train_loader:
        x_batch = x_batch.to(device, non_blocking=True)
        y_batch = y_batch.to(device, non_blocking=True)
        y_batch_train = smooth_labels(y_batch, label_smoothing)

        optimizer.zero_grad(set_to_none=True)
        with torch.autocast(device_type=device.type, dtype=torch.float16, enabled=use_amp):
            logits = model(x_batch)
        # Loss computed in float32, outside autocast, for numerical stability
        loss = criterion(logits.float(), y_batch_train)
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
```

```
            scaler.update()

        # Selection metric: ALWAYS the plain log loss on true (non-smoothed)
        # labels of the INNER split, independent of the training loss.
        inner_preds = np.clip(
            predict_probabilities(model, X_inner, device, batch_size, use_amp),
            CLIP_EPS, 1.0 - CLIP_EPS)
        inner_loss = compute_moa_log_loss(y_inner, inner_preds)
        scheduler.step(inner_loss)

        if inner_loss < best_inner_loss:
            best_inner_loss = inner_loss
            # The best checkpoint is kept in RAM (CPU) and written to disk once at
            # the end, instead of on every improving epoch.
            best_state_dict = {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}
            epochs_without_improvement = 0
        else:
            epochs_without_improvement += 1

        if epochs_without_improvement >= early_stopping_patience:
            if verbose:
                print(f"--> Early stopping {model.name} fold {fold}{checkpoint_suffix} "
                      f"at epoch {epoch + 1}/{epochs} (no improvement for "
                      f"{early_stopping_patience} epochs)")
            break

    # Step 7.3.5 - guard: no finite loss was ever recorded (e.g. NaN divergence)
    if best_state_dict is None:
        raise RuntimeError(f"{model.name} fold {fold}{checkpoint_suffix}: "
                           f"no valid checkpoint (inner loss never finite)")

    # Step 7.3.6 - score the selected checkpoint on the OUTER validation fold, once
    model.load_state_dict(best_state_dict)
    val_preds = np.clip(
        predict_probabilities(model, X_val, device, batch_size, use_amp),
        CLIP_EPS, 1.0 - CLIP_EPS)
    val_loss = compute_moa_log_loss(y_val, val_preds)

    if verbose:
        print(f"--> Outer val loss {model.name} fold {fold}{checkpoint_suffix}: {val_loss:.5f}")
    torch.save(best_state_dict, checkpoint_path)

    # This process's CUDA allocator cache is NOT released automatically when the
    # function returns, only when the PROCESS ends. With several loky workers
    # sharing the same GPU, a cache left behind by one job is not visible to its
    # sibling worker: clearing it explicitly returns it to the GPU's free pool.
    del model, optimizer, scaler, train_loader
    if device.type == "cuda":
        torch.cuda.empty_cache()

    return val_preds, val_loss
```

8. Per-fold preprocessing pipelines

Definition of two types of data transformation: one with PCA (for ResNet) and the other without PCA (for FT-Transformer and CatBoost). A helper removes cached pipeline files so that a new run never reuses artefacts fitted on different splits.

In [9]:

```
# Step 8.1 - remove stale cached pipelines
def clear_cached_pipelines(work_dir=WORK_DIR):
    """Deletes fold pipeline files left by previous runs (possibly fitted on
    different splits or data)."""
    for path in glob.glob(f"{work_dir}/fold_pipeline_*.joblib"):
        os.remove(path)


# Step 8.2 - PCA pipeline (ResNet)
def fit_fold_pipeline(X_train, X_val, feature_cols, g_cols, c_cols, dummy_cols,
                      fold, work_dir=WORK_DIR, variance_threshold=VARIANCE_THRESHOLD):
    # Variance threshold, fit only on the fold's training split
    selector = VarianceThreshold(threshold=variance_threshold)
    X_train_vt = selector.fit_transform(X_train)
    X_val_vt = selector.transform(X_val)
    support = selector.get_support()
    selected_cols = [c for c, keep in zip(feature_cols, support) if keep]

    # Split continuous vs. dummy columns (dummy = cp_time_*, cp_dose_*)
    dummy_idx = [i for i, c in enumerate(selected_cols) if c in dummy_cols]
    cont_idx = [i for i, c in enumerate(selected_cols) if c not in dummy_cols]

    X_train_cont, X_val_cont = X_train_vt[:, cont_idx], X_val_vt[:, cont_idx]
    X_train_dummy, X_val_dummy = X_train_vt[:, dummy_idx], X_val_vt[:, dummy_idx]
    cont_cols = [selected_cols[i] for i in cont_idx]

    g_idx_in_cont = [i for i, c in enumerate(cont_cols) if c in g_cols]
    c_idx_in_cont = [i for i, c in enumerate(cont_cols) if c in c_cols]

    # RankGauss only on the continuous columns (not on the 0/1 dummies)
    quantile_transformer = QuantileTransformer(
        n_quantiles=N_QUANTILES, output_distribution="normal", random_state=RANDOM_STATE)
    X_train_tf = quantile_transformer.fit_transform(X_train_cont)
    X_val_tf = quantile_transformer.transform(X_val_cont)

    # PCA on the g- and c- blocks
    pca_g = PCA(n_components=PCA_G_COMPONENTS, random_state=RANDOM_STATE)
    train_pca_g = pca_g.fit_transform(X_train_tf[:, g_idx_in_cont])
    val_pca_g = pca_g.transform(X_val_tf[:, g_idx_in_cont])

    pca_c = PCA(n_components=PCA_C_COMPONENTS, random_state=RANDOM_STATE)
    train_pca_c = pca_c.fit_transform(X_train_tf[:, c_idx_in_cont])
    val_pca_c = pca_c.transform(X_val_tf[:, c_idx_in_cont])

    # Final concatenation: transformed continuous + PCA + raw dummies
    X_train_final = np.hstack([X_train_tf, train_pca_g, train_pca_c, X_train_dummy])
    X_val_final = np.hstack([X_val_tf, val_pca_g, val_pca_c, X_val_dummy])

    # Persist the whole fold pipeline in a single file
    joblib.dump(
        {"selector": selector, "quantile transformer": quantile transformer,
```

```
        "pca_g": pca_g, "pca_c": pca_c,
        "cont_idx": cont_idx, "dummy_idx": dummy_idx,
        "g_idx_in_cont": g_idx_in_cont, "c_idx_in_cont": c_idx_in_cont},
       f"{work_dir}/fold_pipeline_{fold}.joblib",
    )
    return X_train_final, X_val_final


def apply_fold_pipeline(X, fold, work_dir=WORK_DIR):
    """Applies the pipeline already fitted for that fold (training or
    inference alike: a pure transform, no fitting)."""
    artifacts = joblib.load(f"{work_dir}/fold_pipeline_{fold}.joblib")
    X_vt = artifacts["selector"].transform(X)
    X_cont = X_vt[:, artifacts["cont_idx"]]
    X_dummy = X_vt[:, artifacts["dummy_idx"]]
    X_tf = artifacts["quantile_transformer"].transform(X_cont)
    pca_g_out = artifacts["pca_g"].transform(X_tf[:, artifacts["g_idx_in_cont"]])
    pca_c_out = artifacts["pca_c"].transform(X_tf[:, artifacts["c_idx_in_cont"]])
    return np.hstack([X_tf, pca_g_out, pca_c_out, X_dummy])
```

In [10]:

```
# Step 8.3 - raw pipeline, no PCA (FT-Transformer, CatBoost)
def fit_fold_pipeline_raw(X_train, X_val, feature_cols, dummy_cols,
                          fold, work_dir=WORK_DIR, variance_threshold=VARIANCE_THRESHOLD,
                          reuse_if_cached=False):
    """"Raw" pipeline for tree-based models (and for the FT-Transformer,
    which reuses it to get one token per original feature, without PCA):
    VarianceThreshold + RankGauss on the continuous columns, NO PCA.

    PCA rotates the axes towards directions of maximum global variance:
    this helps dense networks (which combine all inputs in every neuron)
    but strips features of their original per-feature meaning, which is
    what a tree split relies on (a split on a PCA component is an oblique
    hyperplane in the original space, rarely aligned with a single gene or
    cell-viability column). The dummy columns (cp_time/cp_dose one-hot) are
    left raw, as in the PCA pipeline.

    CACHE: with reuse_if_cached=True and an existing .joblib file for this
    fold, the stored pipeline is applied in transform-only mode instead of
    being refitted. This is only safe within a single run on the same splits
    (the CatBoost CV does it after the neural-network CV). The default is
    False, and clear_cached_pipelines() is called at the start of each run,
    so artefacts from a previous run are never reused silently."""
    cache_path = f"{work_dir}/fold_pipeline_raw_{fold}.joblib"
    if reuse_if_cached and os.path.exists(cache_path):
        return (apply_fold_pipeline_raw(X_train, fold, work_dir=work_dir),
                apply_fold_pipeline_raw(X_val, fold, work_dir=work_dir))

    selector = VarianceThreshold(threshold=variance_threshold)
    X_train_vt = selector.fit_transform(X_train)
    X_val_vt = selector.transform(X_val)
    support = selector.get_support()
    selected_cols = [c for c, keep in zip(feature_cols, support) if keep]
```

```
    dummy_idx = [i for i, c in enumerate(selected_cols) if c in dummy_cols]
    cont_idx = [i for i, c in enumerate(selected_cols) if c not in dummy_cols]

    X_train_cont, X_val_cont = X_train_vt[:, cont_idx], X_val_vt[:, cont_idx]
    X_train_dummy, X_val_dummy = X_train_vt[:, dummy_idx], X_val_vt[:, dummy_idx]

    quantile_transformer = QuantileTransformer(
        n_quantiles=N_QUANTILES, output_distribution="normal", random_state=RANDOM_STATE)
    X_train_tf = quantile_transformer.fit_transform(X_train_cont)
    X_val_tf = quantile_transformer.transform(X_val_cont)

    joblib.dump(
        {"selector": selector, "quantile_transformer": quantile_transformer,
         "cont_idx": cont_idx, "dummy_idx": dummy_idx},
        cache_path,
    )

    X_train_final = np.hstack([X_train_tf, X_train_dummy])
    X_val_final = np.hstack([X_val_tf, X_val_dummy])
    return X_train_final, X_val_final


def apply_fold_pipeline_raw(X, fold, work_dir=WORK_DIR):
    artifacts = joblib.load(f"{work_dir}/fold_pipeline_raw_{fold}.joblib")
    X_vt = artifacts["selector"].transform(X)
    X_cont = X_vt[:, artifacts["cont_idx"]]
    X_dummy = X_vt[:, artifacts["dummy_idx"]]
    X_tf = artifacts["quantile_transformer"].transform(X_cont)
    return np.hstack([X_tf, X_dummy])
```

9. Cross-validation: neural network ensemble (ResNet, FT-Transformer)

GPU multiprocessing orchestration via `joblib` (`loky` backend): 30 jobs (2 models × 3 seeds × 5 folds) are scheduled on a pool of `N_JOBS_NN` parallel workers (2 per GPU).

In [11]:

```
# Step 9.1 - model registry and helpers
MODEL_CLASSES = {"resnet": TabularResNet, "ft_transformer": FTTransformer}
MODEL_NAMES = tuple(MODEL_CLASSES.keys())


def shutdown_worker_pool():
    """Shuts down the loky worker pool so that idle workers release their CUDA
    memory. The informational loky warning "A worker stopped while some jobs
    were given to the executor" is raised by this shutdown itself, when all jobs
    are already done, so only that specific message is silenced here."""
    with warnings.catch_warnings():
        warnings.filterwarnings(
            "ignore", message="A worker stopped while some jobs were given")
        get_reusable_executor().shutdown(wait=True)
```

```python
def _build_model(model_name, input_dim, output_dim):
    return MODEL_CLASSES[model_name](input_dim=input_dim, output_dim=output_dim)


def _resolve_device(gpu_id):
    """Single place for the GPU-assignment logic: jobs are distributed
    round-robin across all available GPUs (rather than converging on a
    single one), so both T4s are used evenly."""
    if gpu_id is not None and N_GPUS > 0:
        torch.cuda.set_device(gpu_id)
        return torch.device(f"cuda:{gpu_id}")
    return torch.device("cpu")


# Step 9.2 - single training job (runs in a worker process)
def _train_nn_job(fold, model_name, seed, X_train, y_train, X_val, y_val, gpu_id,
                  batch_size=None):
    """
    Runs in a worker process (joblib/loky). Receives features that have
    already been preprocessed with the pipeline appropriate for this model:
    PCA features for resnet, raw RankGauss features (no PCA, one token
    per original feature) for the FT-Transformer.
    """
    device = _resolve_device(gpu_id)

    # Seed BEFORE building the model, so the initial weights depend on the seed
    set_global_seed(seed)
    model = _build_model(model_name, input_dim=X_train.shape[1], output_dim=y_train.shape[1])

    job_batch_size = batch_size or BATCH_SIZE_BY_MODEL.get(model_name, BATCH_SIZE)
    val_preds, val_loss = train_fold(
        fold, model, X_train, y_train, X_val, y_val,
        device=device, seed=seed, batch_size=job_batch_size,
        checkpoint_suffix=f"_seed{seed}",
    )
    return model_name, fold, seed, val_preds, val_loss


# Step 9.3 - full neural-network cross-validation
def run_cross_validation(X, Y, feature_cols, g_cols, c_cols, dummy_cols,
                         n_folds=N_FOLDS, seeds=SEEDS, n_jobs=N_JOBS_NN,
                         model_names=MODEL_NAMES):
    mskf = MultilabelStratifiedKFold(n_splits=n_folds, shuffle=True, random_state=RANDOM_STATE)
    splits = list(mskf.split(X, Y))

    oof = {name: np.zeros_like(Y, dtype=np.float64) for name in model_names}

    # Step 9.3.1 - per-fold preprocessing, run sequentially (CPU-bound and
    # already fast). Two variants:
    #   - "pca": RankGauss + PCA (resnet)               -> fit_fold_pipeline
    #   - "raw": RankGauss WITHOUT PCA (ft_transformer) -> fit_fold_pipeline_raw
    # Both are fitted from scratch here (reuse_if_cached=False); the CatBoost CV
    # later reuses the "raw" files written now.
    fold_data = {}
```

```
for fold, (train_idx, val_idx) in enumerate(splits):
    print(f"Preprocessing fold {fold}...")
    X_train, y_train = X[train_idx], Y[train_idx]
    X_val, y_val = X[val_idx], Y[val_idx]

    X_train_pca, X_val_pca = fit_fold_pipeline(
        X_train, X_val, feature_cols, g_cols, c_cols, dummy_cols, fold
    )
    X_train_raw, X_val_raw = fit_fold_pipeline_raw(
        X_train, X_val, feature_cols, dummy_cols, fold
    )
    fold_data[fold] = {
        "pca": (X_train_pca, X_val_pca),
        "raw": (X_train_raw, X_val_raw),
        "y_train": y_train, "y_val": y_val, "val_idx": val_idx,
    }

# Step 9.3.2 - job list: one job per (fold, model, seed) = 30 jobs, executed
# by n_jobs workers at a time, assigned round-robin across the GPUs. Each
# job explicitly receives its own fold's preprocessed features.
jobs = [(fold, model_name, seed)
        for fold in range(n_folds)
        for model_name in model_names
        for seed in seeds]

def _job_arrays(fold, model_name):
    key = "raw" if model_name == "ft_transformer" else "pca"
    X_train_final, X_val_final = fold_data[fold][key]
    return X_train_final, fold_data[fold]["y_train"], X_val_final, fold_data[fold]["y_val"]

print(f"Launching {len(jobs)} training jobs "
      f"({len(model_names)} models x {len(seeds)} seeds x {n_folds} folds) "
      f"on {n_jobs} parallel workers...")

results = Parallel(n_jobs=n_jobs, backend="loky")(
    delayed(_train_nn_job)(
        fold, model_name, seed, *_job_arrays(fold, model_name),
        gpu_id=(job_number % N_GPUS) if N_GPUS > 0 else None,
    )
    for job_number, (fold, model_name, seed) in enumerate(jobs)
)

# Explicitly shuts down the loky worker pool. torch does not release a
# process's CUDA allocator cache when a job finishes, only when the PROCESS
# ends: without this shutdown, memory would stay occupied by idle workers
# while the next stages start, increasing the risk of OOM.
shutdown_worker_pool()

# Step 9.3.3 - aggregate: average over seeds for each (fold, model) pair and
# print one concise summary line per pair instead of 30 raw per-job lines.
seed_preds = {(fold, name): [] for fold in range(n_folds) for name in model_names}
seed_losses = {(fold, name): [] for fold in range(n_folds) for name in model_names}
for model_name, fold, seed, job_preds, job_loss in results:
    seed_preds[(fold, model_name)].append(job_preds)
```

```
        seed_losses[(fold, model_name)].append(job_loss)

    for fold in range(n_folds):
        val_idx = fold_data[fold]["val_idx"]
        for name in model_names:
            oof[name][val_idx] = np.mean(seed_preds[(fold, name)], axis=0)
            mean_loss = np.mean(seed_losses[(fold, name)])
            print(f"  fold {fold} | {name:14s} | mean val loss over {len(seeds)} seeds: {mean_loss:.5f}")

    return oof, splits
```

In [12]:

```
# Step 9.4 - single test-set inference job (runs in a worker process)
def _predict_nn_job(fold, model_name, seed, X_test_final, num_targets, gpu_id,
                    work_dir=WORK_DIR, batch_size=None):
    device = _resolve_device(gpu_id)

    model = _build_model(model_name, X_test_final.shape[1], num_targets)
    checkpoint_path = f"{work_dir}/best_{model.name}_fold_{fold}_seed{seed}.pth"
    model.load_state_dict(torch.load(checkpoint_path, map_location=device, weights_only=True))
    model = model.to(device).eval()

    # Inference is run in mini-batches rather than a single monolithic forward
    # pass: for the FT-Transformer, one attention matrix of shape
    # (n_test_rows x heads x seq_len x seq_len) with seq_len ~890 would need
    # tens of GB in a single pass, far beyond a T4's VRAM.
    inference_batch_size = batch_size or INFERENCE_BATCH_SIZE_BY_MODEL.get(model_name, BATCH_SIZE)
    test_preds = predict_probabilities(model, X_test_final, device, inference_batch_size,
                                       use_amp=False)

    del model
    if device.type == "cuda":
        torch.cuda.empty_cache()
    return model_name, test_preds


# Step 9.5 - test-set inference, averaged over folds and seeds
def predict_test_set_nn(X_test_final_by_fold_pca, X_test_final_by_fold_raw, num_targets,
                        n_folds=N_FOLDS, seeds=SEEDS, n_jobs=N_JOBS_NN, model_names=MODEL_NAMES):
    """
    Averages test-set predictions across all folds and seeds for each
    neural-network model. Each model receives the test features from the
    preprocessing variant it was trained on: PCA features for resnet,
    raw RankGauss features (no PCA) for the FT-Transformer.
    """
    n_samples = next(iter(X_test_final_by_fold_pca.values())).shape[0]
    preds = {name: np.zeros((n_samples, num_targets)) for name in model_names}
    n_checkpoints = n_folds * len(seeds)

    jobs = [(fold, model_name, seed)
            for fold in range(n_folds)
            for model_name in model_names
            for seed in seeds]
```

```
    def _test_features(fold, model_name):
        return (X_test_final_by_fold_raw[fold] if model_name == "ft_transformer"
                else X_test_final_by_fold_pca[fold])

    results = Parallel(n_jobs=n_jobs, backend="loky")(
        delayed(_predict_nn_job)(
            fold, model_name, seed, _test_features(fold, model_name), num_targets,
            gpu_id=(job_number % N_GPUS) if N_GPUS > 0 else None,
        )
        for job_number, (fold, model_name, seed) in enumerate(jobs)
    )
    shutdown_worker_pool()

    for model_name, job_preds in results:
        preds[model_name] += job_preds / n_checkpoints

    return preds
```

10. Third model: CatBoost on raw (non-PCA) features

CatBoost model training, optimizing all targets at once with `MultiLogloss`.

In [13]:

```
# Step 10.1 - CatBoost training on one fold
def train_catboost_fold(fold, X_train, y_train, X_val, y_val, work_dir=WORK_DIR,
                        iterations=N_ESTIMATORS_CATBOOST, learning_rate=CATBOOST_LEARNING_RATE,
                        depth=CATBOOST_DEPTH, gpu_id=0):
    from catboost import CatBoostClassifier

    num_targets = y_train.shape[1]
    # Some of the 206 MoA targets have very few positives: in certain folds
    # a target's training column can be ALL zero. CatBoost's MultiLogloss
    # requires both classes to be present for EVERY target passed to fit()
    # ("All train targets are equal"). Only non-constant targets are
    # included in the fit; for the others, the constant value observed in
    # the training fold (almost always 0.0) is used directly.
    col_is_constant = np.array([len(np.unique(y_train[:, t])) < 2 for t in range(num_targets)])
    active_cols = np.where(~col_is_constant)[0]
    constant_cols = np.where(col_is_constant)[0]
    constant_vals = y_train.mean(axis=0)  # only valid at the constant_cols positions

    if len(constant_cols) > 0:
        print(f"[catboost | fold {fold}] {len(constant_cols)} constant target(s) in "
              f"the training fold (excluded from the fit, using the observed constant)")

    model = CatBoostClassifier(
        loss_function="MultiLogloss",   # a single fit over the active targets,
        iterations=iterations,          # rather than N separate binary fits
        learning_rate=learning_rate,
        depth=depth,
        task_type="GPU" if N_GPUS > 0 else "CPU",
        devices=str(gpu_id) if N_GPUS > 0 else None,
```

```
        random_seed=RANDOM_STATE,
        verbose=False,
    )
    # No eval_set / early stopping: fixed number of iterations, so the outer
    # validation fold is never used for model selection.
    model.fit(X_train, y_train[:, active_cols])

    val_preds = np.zeros((X_val.shape[0], num_targets))
    if len(active_cols) > 0:
        val_preds[:, active_cols] = model.predict_proba(X_val)
    if len(constant_cols) > 0:
        val_preds[:, constant_cols] = constant_vals[constant_cols]
    val_preds = np.clip(val_preds, CLIP_EPS, 1 - CLIP_EPS)
    val_loss = compute_moa_log_loss(y_val, val_preds)

    model.save_model(f"{work_dir}/catboost_fold_{fold}.cbm")
    joblib.dump(
        {"active_cols": active_cols, "constant_cols": constant_cols, "constant_vals": constant_vals},
        f"{work_dir}/catboost_fold_{fold}_meta.joblib",
    )
    print(f"[catboost | fold {fold}] val_loss={val_loss:.5f}")
    return val_preds, val_loss


# Step 10.2 - CatBoost cross-validation
def run_cross_validation_catboost(X, Y, feature_cols, dummy_cols, splits, work_dir=WORK_DIR):
    """Reuses the SAME splits as run_cross_validation but a different
    preprocessing pipeline: RankGauss without PCA. Runs sequentially per fold
    rather than through joblib multiprocessing: a multi-output CatBoost fit on
    GPU is already fast, and staying single-process avoids GPU contention."""
    oof_catboost = np.zeros_like(Y, dtype=np.float64)

    for fold, (train_idx, val_idx) in enumerate(splits):
        X_train_raw, y_train = X[train_idx], Y[train_idx]
        X_val_raw, y_val = X[val_idx], Y[val_idx]

        # reuse_if_cached=True: the raw pipelines were fitted moments ago, on the
        # very same splits, by run_cross_validation (same run).
        X_train_final, X_val_final = fit_fold_pipeline_raw(
            X_train_raw, X_val_raw, feature_cols, dummy_cols, fold,
            work_dir=work_dir, reuse_if_cached=True
        )
        val_preds, _ = train_catboost_fold(
            fold, X_train_final, y_train, X_val_final, y_val,
            gpu_id=(fold % N_GPUS) if N_GPUS > 0 else 0, work_dir=work_dir,
        )
        oof_catboost[val_idx] = val_preds

    return oof_catboost


# Step 10.3 - CatBoost test-set inference
def predict_test_set_catboost(X_test_raw, num_targets, n_folds=N_FOLDS, work_dir=WORK_DIR):
    from catboost import CatBoostClassifier
```

```
    n_samples = X_test_raw.shape[0]
    preds = np.zeros((n_samples, num_targets))
    for fold in range(n_folds):
        X_test_final = apply_fold_pipeline_raw(X_test_raw, fold, work_dir=work_dir)
        meta = joblib.load(f"{work_dir}/catboost_fold_{fold}_meta.joblib")
        active_cols, constant_cols, constant_vals = (
            meta["active_cols"], meta["constant_cols"], meta["constant_vals"]
        )

        model = CatBoostClassifier()
        model.load_model(f"{work_dir}/catboost_fold_{fold}.cbm")

        fold_preds = np.zeros((n_samples, num_targets))
        if len(active_cols) > 0:
            fold_preds[:, active_cols] = model.predict_proba(X_test_final)
        if len(constant_cols) > 0:
            fold_preds[:, constant_cols] = constant_vals[constant_cols]

        preds += np.clip(fold_preds, CLIP_EPS, 1 - CLIP_EPS) / n_folds
    return preds
```

11. Ensemble blending

Computation of the optimal weights for combining the three models, and evaluation of the blend with a nested approach.

In [14]:

```
# Step 11.1 - blend weight optimisation
def optimize_blend_weights(oof_list, Y):
    """
    Optimises the blend weights on the simplex (weights >= 0, sum = 1)
    via a softmax reparameterisation: free logits are optimised (no
    explicit constraint), then softmax is applied to obtain the weights.
    Nelder-Mead is used instead of SLSQP with an explicit equality
    constraint: on an objective averaged over millions of elements
    (N x 206 targets), SLSQP's finite-difference gradient can appear
    numerically zero at iteration 0, causing the optimiser to terminate
    WITHOUT moving from x0 and WITHOUT raising an error (observed result:
    weights all exactly equal to 1/n_models). Nelder-Mead is derivative-free
    and robust for a low-dimensional problem like this one.
    """
    n_models = len(oof_list)

    def objective(logits):
        blend_weights = np.exp(logits - logits.max())
        blend_weights = blend_weights / blend_weights.sum()
        blend = sum(weight * oof_pred for weight, oof_pred in zip(blend_weights, oof_list))
        return compute_moa_log_loss(Y, blend)

    x0 = np.zeros(n_models)  # uniform logits -> softmax(x0) = 1/n_models
    result = minimize(objective, x0=x0, method="Nelder-Mead",
                      options={"xatol": 1e-6, "fatol": 1e-9, "maxiter": 3000})
```

```
    if not result.success:
        print(f"[blend] WARNING: optimisation did not converge cleanly "
              f"({result.message}); using the best point found regardless.")

    blend_weights = np.exp(result.x - result.x.max())
    return blend_weights / blend_weights.sum()


# Step 11.2 - nested validation of the blend
def nested_blend_validation(oof_list, Y, splits):
    """
    Honest estimate of the blending step: for each evaluation fold, the blend
    weights are optimised ONLY on the other folds, never on the data the score
    is measured on. This avoids overestimating the blend's gain.
    """
    fold_scores, fold_weights = [], []

    for val_fold_idx, (_, val_idx) in enumerate(splits):
        train_mask = np.ones(Y.shape[0], dtype=bool)
        train_mask[val_idx] = False

        oof_train = [oof_pred[train_mask] for oof_pred in oof_list]
        oof_val = [oof_pred[val_idx] for oof_pred in oof_list]

        fold_blend_weights = optimize_blend_weights(oof_train, Y[train_mask])
        blend_val = sum(weight * oof_pred for weight, oof_pred in zip(fold_blend_weights, oof_val))
        score = compute_moa_log_loss(Y[val_idx], blend_val)

        fold_scores.append(score)
        fold_weights.append(fold_blend_weights)
        print(f"[nested blend] fold {val_fold_idx}: score={score:.5f} "
              f"weights={np.round(fold_blend_weights, 3)}")

    mean_score, std_score = float(np.mean(fold_scores)), float(np.std(fold_scores))
    final_weights = np.mean(fold_weights, axis=0)
    final_weights = final_weights / final_weights.sum()

    print(f"Honest nested-blend score: {mean_score:.5f} +/- {std_score:.5f}")
    print(f"Final weights (nested mean): {np.round(final_weights, 4)}")
    return final_weights, mean_score, std_score
```

12. Experiment logging

Definition of a function that saves all of the hyperparameters and the experiment metrics in JSON format, adding also a timestamp.

In [15]:

```python
# Step 12.1 - JSON experiment log
def log_experiment(run_name, params, metrics, log_dir=LOG_DIR):
    os.makedirs(log_dir, exist_ok=True)
    record = {
        "run_name": run_name,
        "timestamp": time.strftime("%Y-%m-%d %H:%M:%S"),
        "params": params,
        "metrics": metrics,
    }
    path = os.path.join(log_dir, f"{run_name}_{int(time.time())}.json")
    with open(path, "w") as log_file:
        json.dump(record, log_file, indent=2)
    print(f"Experiment log saved: {path}")
```

13. Submission building and sanity checks

Definition of the functions that build and validate `submission.csv`. Control rows (`cp_type == "ctl_vehicle"`) are set to 0.0 as per the competition rules, because by definition they have no mechanism of action.

In [16]:

```python
# Step 13.1 - build the submission DataFrame
def build_submission(raw_test, sample_submission, target_cols, trt_preds, trt_mask):
    """
    Places the model predictions on the treated test rows and explicitly sets
    the control-vehicle rows to 0.0 (they never have a mechanism of action),
    which lowers the log loss.
    """
    # Row order and columns of the test set and of the sample submission must match
    assert (sample_submission["sig_id"].values == raw_test["sig_id"].values).all(), \
        "sample_submission and test_features are not in the same row order"
    assert list(sample_submission.columns[1:]) == list(target_cols), \
        "Target columns differ from the sample submission"

    n_samples = len(raw_test)
    num_targets = len(target_cols)

    full_preds = np.zeros((n_samples, num_targets), dtype=np.float64)
    trt_indices = raw_test[trt_mask].index.values
    full_preds[trt_indices] = trt_preds
    full_preds = np.clip(full_preds, CLIP_EPS, 1.0 - CLIP_EPS)

    submission = sample_submission.copy()
    submission[target_cols] = full_preds
    submission.loc[raw_test["cp_type"] == CTL_LABEL, target_cols] = 0.0
    return submission


# Step 13.2 - sanity checks before export
def sanity_check_submission(submission, sample_submission, raw_test, target_cols):
    """
    Validates shape and columns, checks for NaN and infinite values, and
    ensures the control rows are exactly 0.0 before export, preventing
    submission failures on Kaggle.
    """
    assert submission.shape == sample_submission.shape, f"Wrong shape: {submission.shape}"
    assert list(submission.columns) == list(sample_submission.columns), "Column mismatch"
    assert submission.isna().sum().sum() == 0, "NaNs present"
    assert np.isinf(submission[target_cols].values).sum() == 0, "Infinite values present"

    ctl_ids = raw_test.loc[raw_test["cp_type"] == CTL_LABEL, "sig_id"]
    ctl_preds = submission[submission["sig_id"].isin(ctl_ids)][target_cols].values
    assert ctl_preds.max() == 0.0, f"ctl_vehicle rows not zeroed: max={ctl_preds.max()}"

    trt_preds = submission[~submission["sig_id"].isin(ctl_ids)][target_cols].values
    print(f"Min trt_cp probability: {trt_preds.min():.2e} | Max: {trt_preds.max():.2e} | Mean: {trt_preds.mean():.5f}")
    print("Sanity check passed.")
```

14. Main pipeline

Execution of all the previous steps in order, to generate the `submission.csv` file in `WORK_DIR`.

In [17]:

```
# Step 14.0 - start from a clean state: no cached pipelines from previous runs
clear_cached_pipelines(WORK_DIR)

# Step 14.1 - load and clean data; drop control rows from training
raw_features, raw_targets, raw_test, sample_submission = load_raw_data(DATA_DIR)
features, targets = clean_and_align(raw_features, raw_targets)

trt_mask_train = features["cp_type"] != CTL_LABEL
features = features[trt_mask_train].reset_index(drop=True)
targets = targets[trt_mask_train].reset_index(drop=True)

# Step 14.2 - feature engineering
g_cols = [c for c in features.columns if c.startswith("g-")]
c_cols = [c for c in features.columns if c.startswith("c-")]

features = add_row_statistics(features, g_cols, c_cols)
features = encode_categoricals(features, dummy_cols=["cp_time", "cp_dose"])

feature_cols = [c for c in features.columns if c not in ("sig_id", "cp_type")]
target_cols = [c for c in targets.columns if c != "sig_id"]
dummy_cols = [c for c in feature_cols if c.startswith("cp_time_") or c.startswith("cp_dose_")]

X = features[feature_cols].values
Y = targets[target_cols].values

# Step 14.3 - neural-network CV: ResNet + FT-Transformer, multi-seed
time_start = time.time()
oof, splits = run_cross_validation(X, Y, feature_cols, g_cols, c_cols, dummy_cols)
time_nn_cv_done = time.time()
for name in MODEL_NAMES:
    print(f"{name:14s} log loss OOF: {compute_moa_log_loss(Y, oof[name]):.5f}")

# Step 14.4 - CatBoost CV, RankGauss features WITHOUT PCA
oof_catboost = run_cross_validation_catboost(X, Y, feature_cols, dummy_cols, splits)
time_catboost_cv_done = time.time()
print(f"catboost       log loss OOF: {compute_moa_log_loss(Y, oof_catboost):.5f}")
print(f"[timing] NN CV: {(time_nn_cv_done - time_start) / 60:.1f} min | "
      f"CatBoost CV: {(time_catboost_cv_done - time_nn_cv_done) / 60:.1f} min")

ALL_MODEL_NAMES = MODEL_NAMES + ("catboost",)
oof["catboost"] = oof_catboost

# Step 14.5 - ensemble blending with nested CV on the weights
oof_list = [oof[name] for name in ALL_MODEL_NAMES]
final_weights, nested_mean, nested_std = nested_blend_validation(oof_list, Y, splits)
weights = dict(zip(ALL_MODEL_NAMES, final_weights))

oof_blend = sum(weights[name] * oof[name] for name in ALL_MODEL_NAMES)
blend_score_naive = compute_moa_log_loss(Y, oof_blend)
print(f"Ensemble log loss OOF (same data used to fit the weights): {blend_score_naive:.5f}")
print(f"Ensemble log loss, nested (honest): {nested_mean:.5f} +/- {nested_std:.5f}")

# Step 14.6 - experiment logging
log_experiment(
```

```
    run_name="moa_ensemble",
    params={"seeds": SEEDS, "n_folds": N_FOLDS, "epochs": EPOCHS, "learning_rate": LR,
            "weight_decay": WEIGHT_DECAY, "batch_size": BATCH_SIZE,
            "batch_size_by_model": BATCH_SIZE_BY_MODEL,
            "pca_g_components": PCA_G_COMPONENTS, "pca_c_components": PCA_C_COMPONENTS,
            "n_quantiles": N_QUANTILES, "variance_threshold": VARIANCE_THRESHOLD,
            "use_focal_loss": USE_FOCAL_LOSS, "label_smoothing": LABEL_SMOOTHING,
            "early_stopping_patience": EARLY_STOPPING_PATIENCE,
            "inner_val_fraction": INNER_VAL_FRACTION,
            "model_names": list(ALL_MODEL_NAMES),
            "catboost_iterations": N_ESTIMATORS_CATBOOST,
            "catboost_learning_rate": CATBOOST_LEARNING_RATE,
            "catboost_depth": CATBOOST_DEPTH},
    metrics={
        **{f"{name}_oof": compute_moa_log_loss(Y, oof[name]) for name in ALL_MODEL_NAMES},
        "blend_oof_naive": blend_score_naive,
        "blend_oof_nested": nested_mean,
        "blend_oof_nested_std": nested_std,
        **{f"w_{name}": float(weights[name]) for name in ALL_MODEL_NAMES},
    },
)

# Step 14.7 - test set: preprocessing and inference for all models
trt_mask_test = raw_test["cp_type"] != CTL_LABEL
test_trt = raw_test[trt_mask_test].reset_index(drop=True)
test_trt = add_row_statistics(test_trt, g_cols, c_cols)
test_trt = encode_categoricals(test_trt, dummy_cols=["cp_time", "cp_dose"],
                               reference_columns=feature_cols)
X_test_raw = test_trt[feature_cols].values

# Two consistent feature variants, matching what each model was trained on
# (see Step 9): PCA features for resnet, raw RankGauss features (no PCA,
# the same pipeline used by CatBoost) for the FT-Transformer.
X_test_final_by_fold_pca = {
    fold: apply_fold_pipeline(X_test_raw, fold) for fold in range(N_FOLDS)
}
X_test_final_by_fold_raw = {
    fold: apply_fold_pipeline_raw(X_test_raw, fold) for fold in range(N_FOLDS)
}

time_inference_start = time.time()
preds_test = predict_test_set_nn(
    X_test_final_by_fold_pca, X_test_final_by_fold_raw, num_targets=Y.shape[1]
)
preds_test["catboost"] = predict_test_set_catboost(X_test_raw, num_targets=Y.shape[1])
final_test_preds = sum(weights[name] * preds_test[name] for name in ALL_MODEL_NAMES)
time_inference_done = time.time()
print(f"[timing] Test-set inference: {(time_inference_done - time_inference_start) / 60:.1f} min | "
      f"Total run: {(time_inference_done - time_start) / 60:.1f} min")

# Step 14.8 - build, check and export the submission
submission = build_submission(raw_test, sample_submission, target_cols,
                              final_test_preds, trt_mask_test)
```

```python
# Final sanity check, run on the object actually written to disk
sanity_check_submission(submission, sample_submission, raw_test, target_cols)

submission_path = f"{WORK_DIR}/submission.csv"
submission.to_csv(submission_path, index=False)
print(f"{submission_path} written successfully.")
```

```
Preprocessing fold 0...
Preprocessing fold 1...
Preprocessing fold 2...
Preprocessing fold 3...
Preprocessing fold 4...
Launching 30 training jobs (2 models x 3 seeds x 5 folds) on 4 parallel workers...
  fold 0 | resnet         | mean val loss over 3 seeds: 0.01686
  fold 0 | ft_transformer | mean val loss over 3 seeds: 0.01693
  fold 1 | resnet         | mean val loss over 3 seeds: 0.01705
  fold 1 | ft_transformer | mean val loss over 3 seeds: 0.01710
  fold 2 | resnet         | mean val loss over 3 seeds: 0.01652
  fold 2 | ft_transformer | mean val loss over 3 seeds: 0.01656
  fold 3 | resnet         | mean val loss over 3 seeds: 0.01679
  fold 3 | ft_transformer | mean val loss over 3 seeds: 0.01688
  fold 4 | resnet         | mean val loss over 3 seeds: 0.01700
  fold 4 | ft_transformer | mean val loss over 3 seeds: 0.01705
resnet         log loss OOF: 0.01632
ft_transformer log loss OOF: 0.01643
[catboost | fold 0] val_loss=0.01707
[catboost | fold 1] 1 constant target(s) in the training fold (excluded from the fit, using the observed constant)
[catboost | fold 1] val_loss=0.01709
[catboost | fold 2] val_loss=0.01676
[catboost | fold 3] 1 constant target(s) in the training fold (excluded from the fit, using the observed constant)
[catboost | fold 3] val_loss=0.01704
[catboost | fold 4] val_loss=0.01704
catboost       log loss OOF: 0.01700
[timing] NN CV: 96.3 min | CatBoost CV: 29.7 min
[nested blend] fold 0: score=0.01603 weights=[0.447 0.374 0.179]
[nested blend] fold 1: score=0.01618 weights=[0.46  0.369 0.17 ]
[nested blend] fold 2: score=0.01570 weights=[0.437 0.373 0.19 ]
[nested blend] fold 3: score=0.01617 weights=[0.428 0.389 0.183]
[nested blend] fold 4: score=0.01622 weights=[0.465 0.37  0.166]
Honest nested-blend score: 0.01606 +/- 0.00019
Final weights (nested mean): [0.4474 0.375  0.1776]
Ensemble log loss OOF (same data used to fit the weights): 0.01606
Ensemble log loss, nested (honest): 0.01606 +/- 0.00019
Experiment log saved: /kaggle/working/experiment_logs/moa_ensemble_1791460623.json
[timing] Test-set inference: 0.5 min | Total run: 128.6 min
Min trt_cp probability: 1.51e-05 | Max: 9.99e-01 | Mean: 0.00372
Sanity check passed.
/kaggle/working/submission.csv written successfully.
```